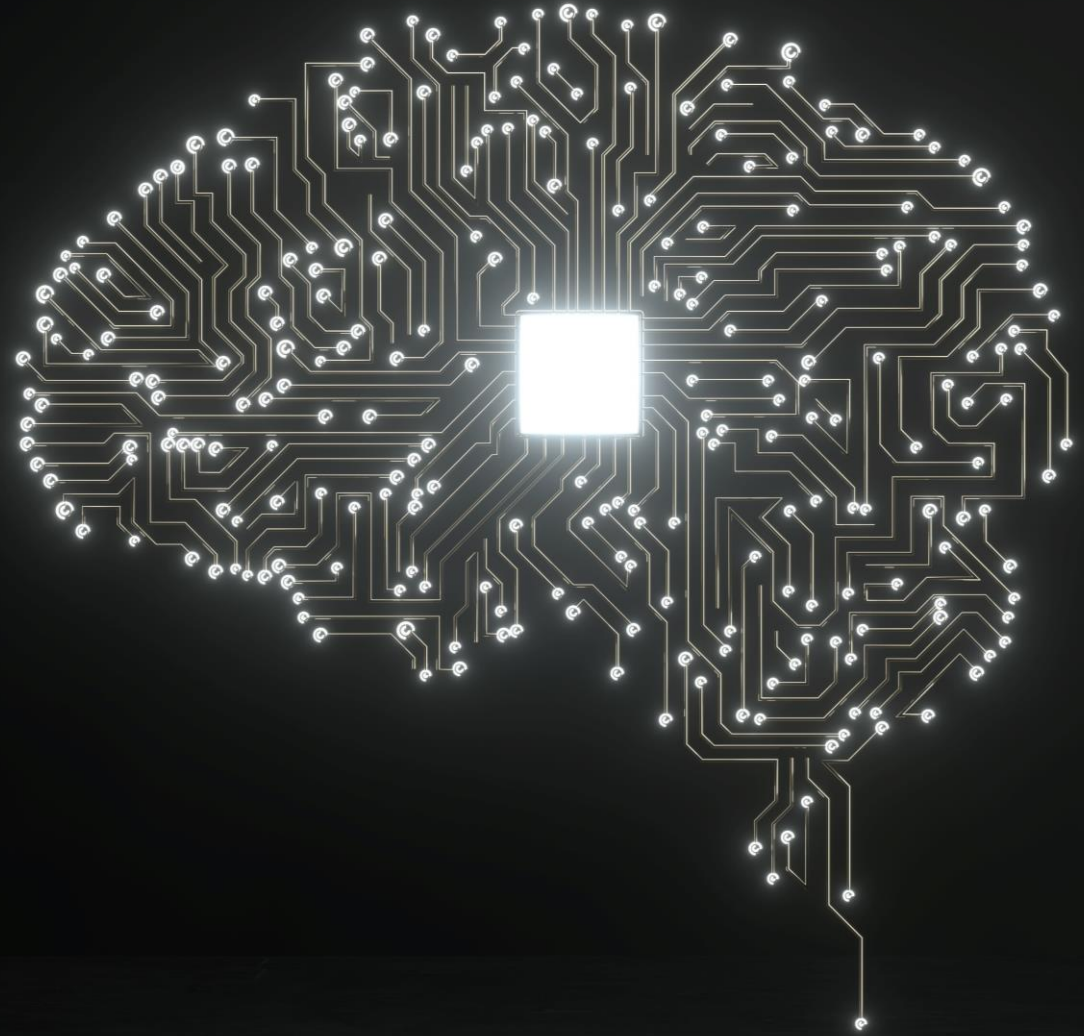


Infrastructure as Code – Configuration Management Part 2

WELCOME TO CLASS 6!



Goals of the day



Ansible Secrets



Ansible Deployments



Ansible at Scale



Photo by [David Becker](#) on [Unsplash](#)

©Maja Bjeletic@unsplash

Agenda

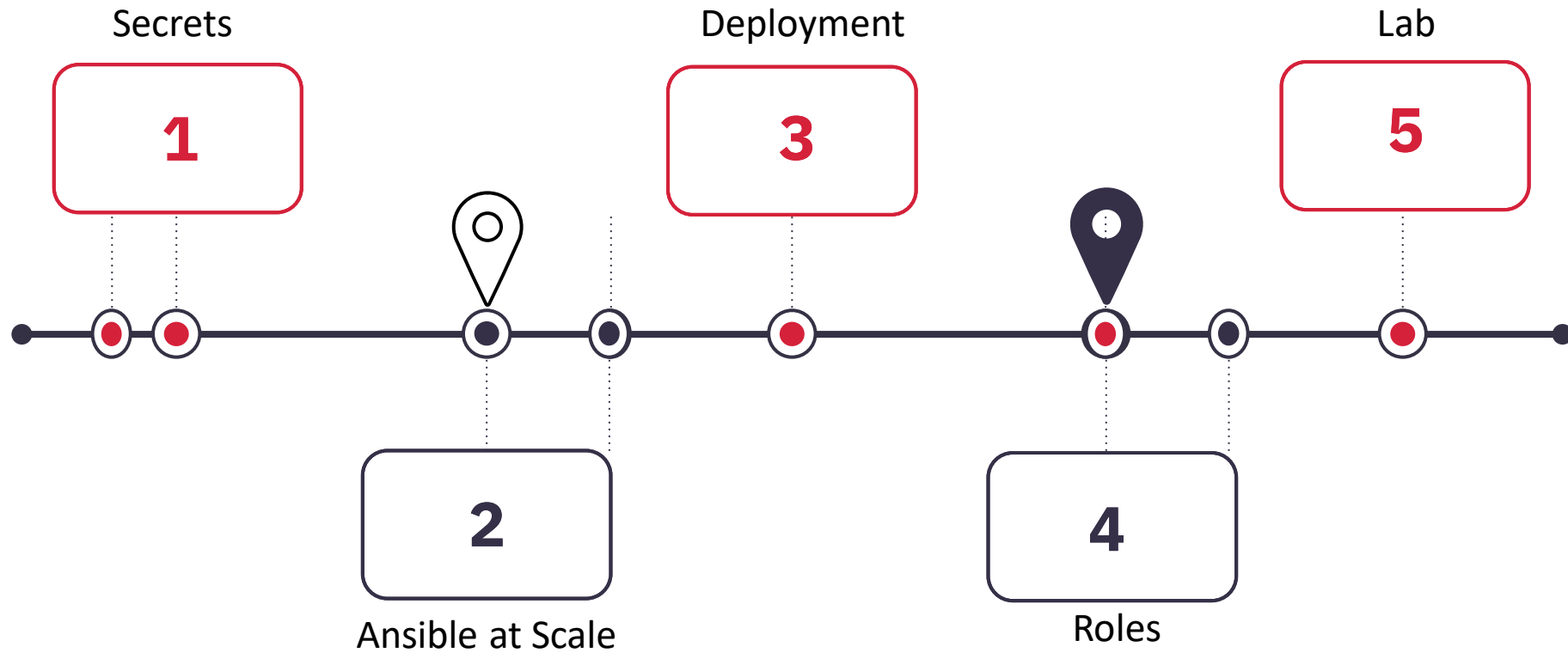
- Secrets management in Ansible
- **Break**
- Managing multiple servers with Ansible
- Using Ansible to deploy code
- Ansible roles
- Lab overview

Photo by David Becker on Unsplash

Review

- How did the exercises from last class go for you?
- Having tried these exercises yourself, what are your thoughts on how Terraform & Ansible interact?
- Was there anything about how we used Ansible that you thought was great, or that you would be concerned about?
- Are there any other use cases for Ansible that you think would be valuable?
- Brief Q & A

Journey Map



Secrets

- You likely noticed that when we created the MySQL instance there was a plain text password in the Ansible playbook.
- This file would normally be checked into git where anyone with access to this repo could read this password.
- This is clearly a bad practice for managing credentials.
- How can we do better with Ansible?

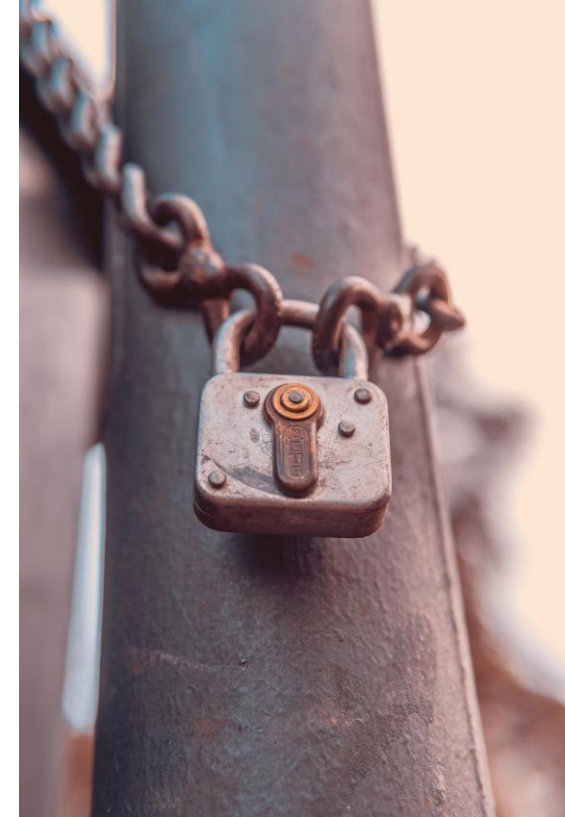


Photo by [Folco Masi](#) on [Unsplash](#)

Secrets – Ansible Vault (1 of 9)

- Ansible comes with a utility called Ansible Vault which can be used to encrypt secrets so that they can be stored in files but are not usable by anyone who does not have a key.
- This key is not checked into git and access to it is limited to a small group of people, typically admins.
- Let's look at how this is used in practice.

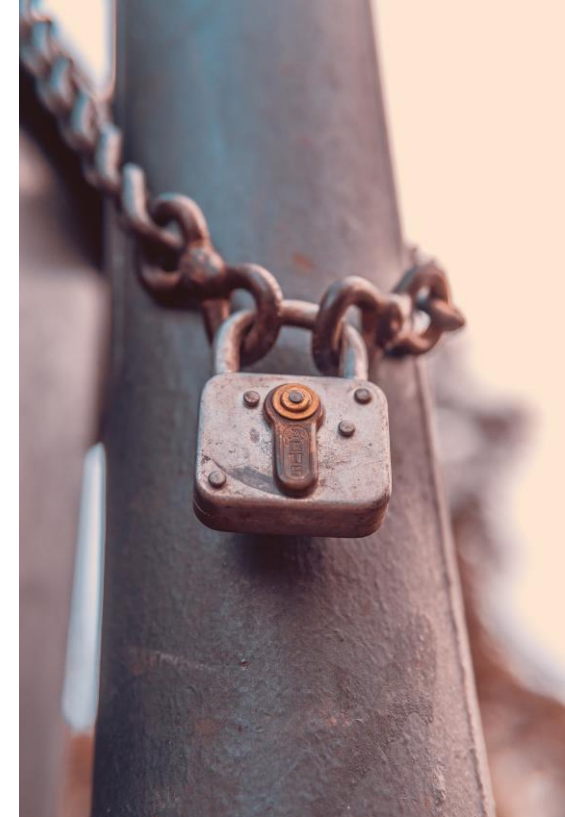


Photo by [Folco Masi](#) on [Unsplash](#)

Secrets – Ansible Vault (2 of 9)

1. First rerun the Maria DB Playbook code from the last class to create a new EC2 instance, with a database and user created using Ansible.
2. SSH to the EC2 and SCP the following files from the repo for this exercise to the home folder of the EC2 user*.
3. Run a `ls -lrt` command and you should see similar output to the screenshot here.

```
[ec2-user@ip-172-31-94-131 ~]$ ls -lrt
total 16
-rw-r--r-- 1 ec2-user ec2-user 830 Jul 21 10:31 install_mariadb.yaml
-rw-r--r-- 1 ec2-user ec2-user 3325 Jul 21 10:31 quotes.sql
-rw-rw-r-- 1 ec2-user ec2-user 28 Jul 21 10:37 vars.yaml
-rw-rw-r-- 1 ec2-user ec2-user 303 Jul 21 10:38 create_dbuser.yaml
```

https://github.com/SMannionYorkU/CloudOps_Module4_Class6_Repo1_AnsibleVault.git

Secrets – Ansible Vault (3 of 9)

4. Review the new files to understand what they will do.

- vars.yaml is a file containing the password for the new user
- create-dbuser.yaml uses the familiar mysql_user module to create a new user Jane (like the new user Bob that we created as part of the last exercise).

This code introduces the concept of variables, which are a powerful way to externalize values from code. Variables allow different values to be used by the code, without the code changing e.g., to configure different environments.

*This is a topic for further reading for you. **

* https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html

```
! vars.yaml x
Class6 > 02.AnsibleVault-Example > ! vars.yaml
1 ---
2 jane_password: "pa44word"
```

```
! create_dbuser.yaml x
Class6 > 02.AnsibleVault-Example > ! create_dbuser.yaml
1 ---
2 - name: Install mariadb
3   hosts: localhost
4   remote_user: ec2-user
5   become: true
6
7   vars_files:
8     - vars.yaml
9
10  tasks:
11    - name: create user with encrypted password
12      mysql_user:
13        name: Jane
14        password: "{{ jane_password }}"
15        priv: ' *.*:ALL '
16        state: present
17
```

Secrets – Ansible Vault (4 of 9)

5. Encrypt the vars.yaml file. Provide a password as your key for the encryption. You will need this password later when you try to use the encrypted file. As you can see, the pa44word string is encrypted, but is still useable as part of our playbook.
6. Run the playbook, adding a switch to request the Vault key and enter the password you provided at encryption time.

```
[ec2-user@ip-172-31-94-131 ~]$ ansible-vault encrypt ./vars.yaml
New Vault password:
Confirm New Vault password:
Encryption successful
[ec2-user@ip-172-31-94-131 ~]$ cat vars.yaml
$ANSIBLE_VAULT;1.1;AES256
31326630323765303739343534366333383737623132326163356464653966363564346134333137
3462303236306638316635653636613562323830396631340a373864316633373334623866303331
35326665633634356465356261323337306230653032616161366131343865666236666463666163
6537653937343638660a336337356635653439363038366333353739643262643531343865326238
33616432643464363139613231376334643430386431646662356236373032636461
```

```
[ec2-user@ip-172-31-94-131 ~]$ ansible-playbook create_dbuser.yaml --ask-vault-pass
Vault password:
[WARNING]: provided hosts list is empty, only localhost is available. Note that
the implicit localhost does not match 'all'

PLAY [Install mariadb] *****
[
TASK [Gathering Facts] *****
ok: [localhost]

TASK [create user with encrypted password] *****
[WARNING]: Module did not set no_log for update_password
changed: [localhost]

PLAY RECAP *****
localhost : ok=2    changed=1    unreachable=0    failed=0    skippe
ed=0      rescued=0    ignored=0
```

Secrets – Ansible Vault (5 of 9)

7. Verify that the new user was created successfully by connecting to MariaDB using the original `pa44word` password that you included in the `vars.yaml` file.

```
[ec2-user@ip-172-31-94-131 ~]$ mysql -u Jane -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 5.5.68-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| bobdata |
| mysql |
| performance_schema |
| test |
+-----+
5 rows in set (0.00 sec)
```

Secrets – Ansible Vault (6 of 9)

7. We can edit the file using the ansible-vault utility to add another password and view it.
8. Edit the create_dbuser.yaml file to change from Jane to Joe.
9. Create a file to hold the password that you have been using for encryption, you will pass this file path as a variable to the ansible-playbook command.

```
[[ec2-user@ip-172-31-94-131 ~]$ ansible-vault view vars.yaml
Vault password:
---
jane_password: pa44word
[[ec2-user@ip-172-31-94-131 ~]$ ansible-vault edit vars.yaml
Vault password:
[[ec2-user@ip-172-31-94-131 ~]$ ansible-vault view vars.yaml
Vault password:
---
jane_password: pa44word
joe_password: pa33word
```

```
---
- name: Install mariadb
  hosts: localhost
  remote_user: ec2-user
  become: true

  vars_files:
    - vars.yaml

  tasks:
    - name: create user with encrypted password
      mysql_user:
        name: Joe
        password: "{{ joe_password }}"
        priv: ' *.*:ALL'
        state: present
```

Secrets – Ansible Vault (7 of 9)

12. Run the playbook again, this time
passing the key file as a parameter. No
further input required.

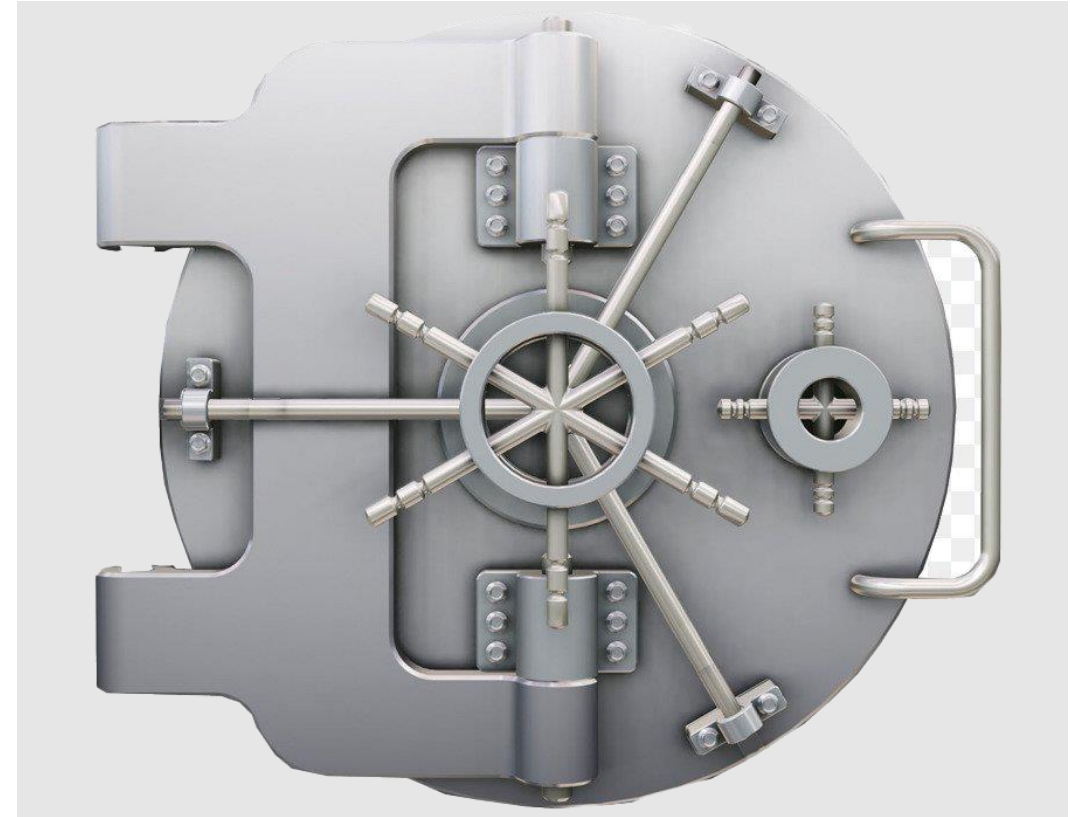
13. Verify that the user Joe was created
successfully with password **pa33word**.

This concludes this exercise.

```
[ec2-user@ip-172-31-94-131 ~]$ ansible-playbook create_dbuser.yaml --vault-password-  
file ./encryption_key.txt  
[WARNING]: provided hosts list is empty, only localhost is available. Note that the  
implicit localhost does not match 'all'  
  
PLAY [Install mariadb] *****  
  
TASK [Gathering Facts] *****  
ok: [localhost]  
  
TASK [create user with encrypted password] *****  
[WARNING]: Module did not set no_log for update_password  
changed: [localhost]  
  
PLAY RECAP *****  
localhost : ok=2 changed=1 unreachable=0 failed=0 skippe  
d=0 rescued=0 ignored=0  
  
[ec2-user@ip-172-31-94-131 ~]$ mysql -u Joe -p  
Enter password:  
Welcome to the MariaDB monitor. Commands end with ; or \g.  
Your MariaDB connection id is 9  
Server version: 5.5.68-MariaDB MariaDB Server  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| bobdata |  
| mysql |  
| performance_schema |  
| test |  
+-----+  
5 rows in set (0.00 sec)  
  
MariaDB [(none)]> exit  
Bye  
[ec2-user@ip-172-31-94-131 ~]$
```


Secrets – Ansible Vault (8 of 9)

- Safe management of secrets is an essential part of a good IaC practice.
- Ansible Vault provides a comprehensive utility for encryption of secrets and is available as a default when Ansible is installed.
- The `encryption_key.txt` file from the previous lab needs to be managed so that access to it is very limited.
- Set the file permissions to 600, so that your user account can read and write to the file, but that others have no access to it.



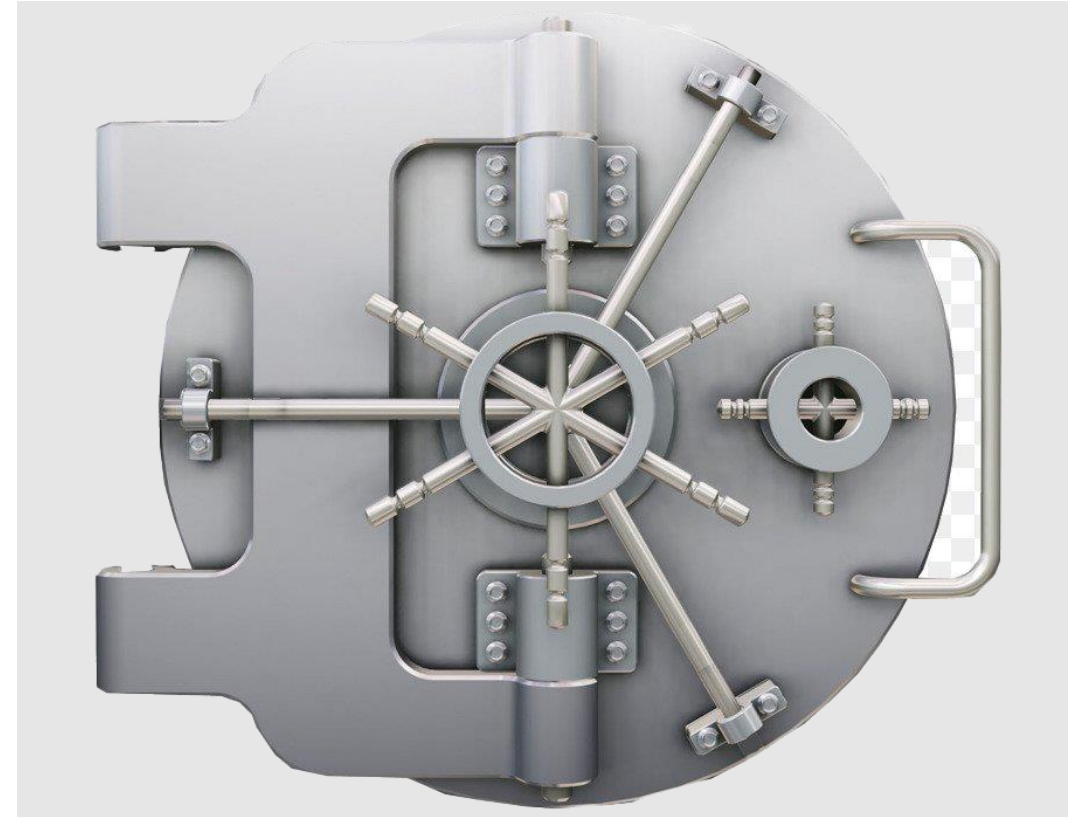
This Photo by Unknown Author is licensed under [CC BY-NC](#)

Secrets – Ansible Vault (9 of 9)

Other options for this capability include Hashicorp Vault, from the creators of Terraform.

Hashicorp Vault centrally manages secrets for the enterprise centrally

- Tokens, which are rotated regularly, are used to control access to application specific Vaults
- Provides an audit trail of how tokens are accessed
- Vault can create dynamic secrets that are not known outside the product, but useable based on token access.



This Photo by Unknown Author is licensed under [CC BY-NC](#)

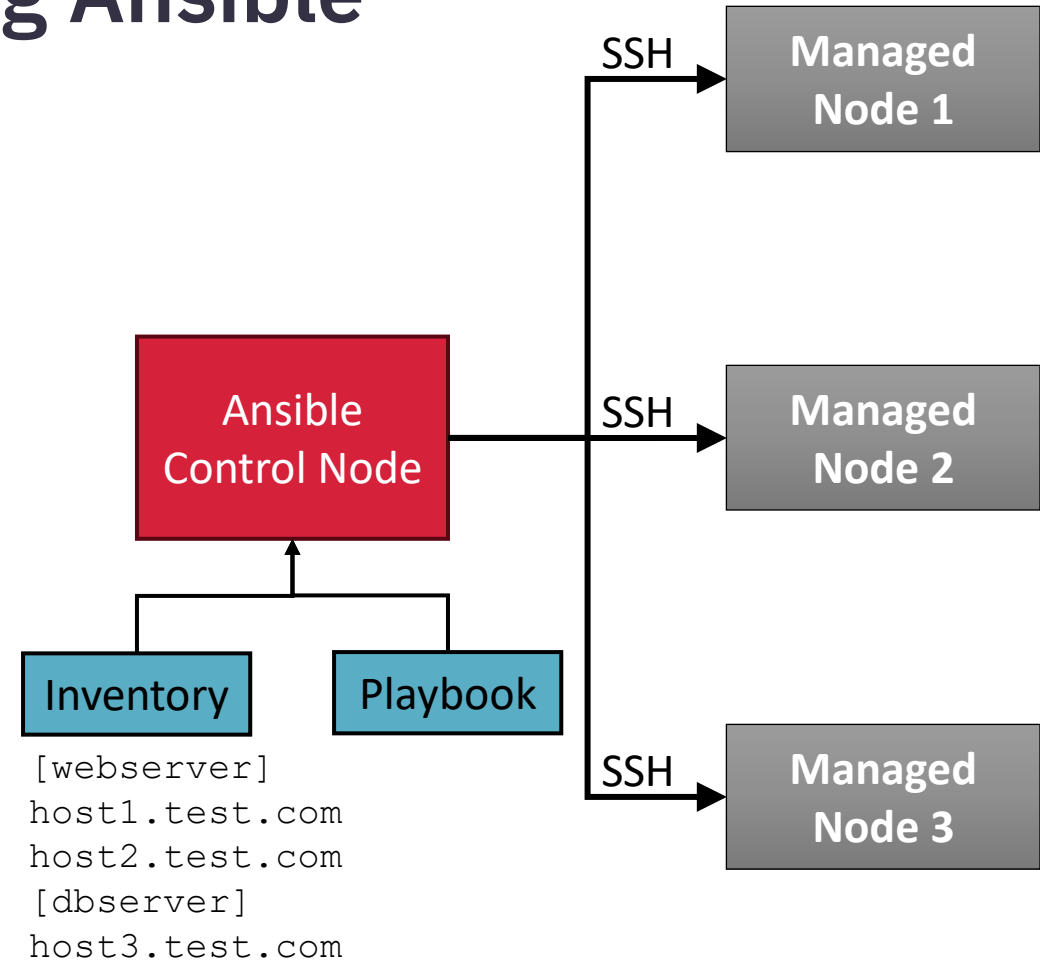
Managing Multiple Servers Using Ansible

Managing Multiple Servers using Ansible (con't)

So far, we have focused all our efforts on running Ansible locally on an EC2 to configure a webserver, a database, manage credentials for that database and learn about secure secrets management.

A more typical use case for Ansible is to use it to manage multiple servers remotely, defined by an inventory file. Fortunately, all the material we have covered is also applicable to the multi-server scenario.

Underpinning this architecture is SSH.



SSH - recap

SSH (or Secure Shell) is used to connect to remote computers so that you can execute commands at the command line.

We have been using this through all of the Terraform and Ansible examples, however for this next exercise we need to understand how this works in more detail.

SSH depends on shared keys to ensure trust between the client and remote servers. These keys are generated on the client side and then copied to the server to establish trust.



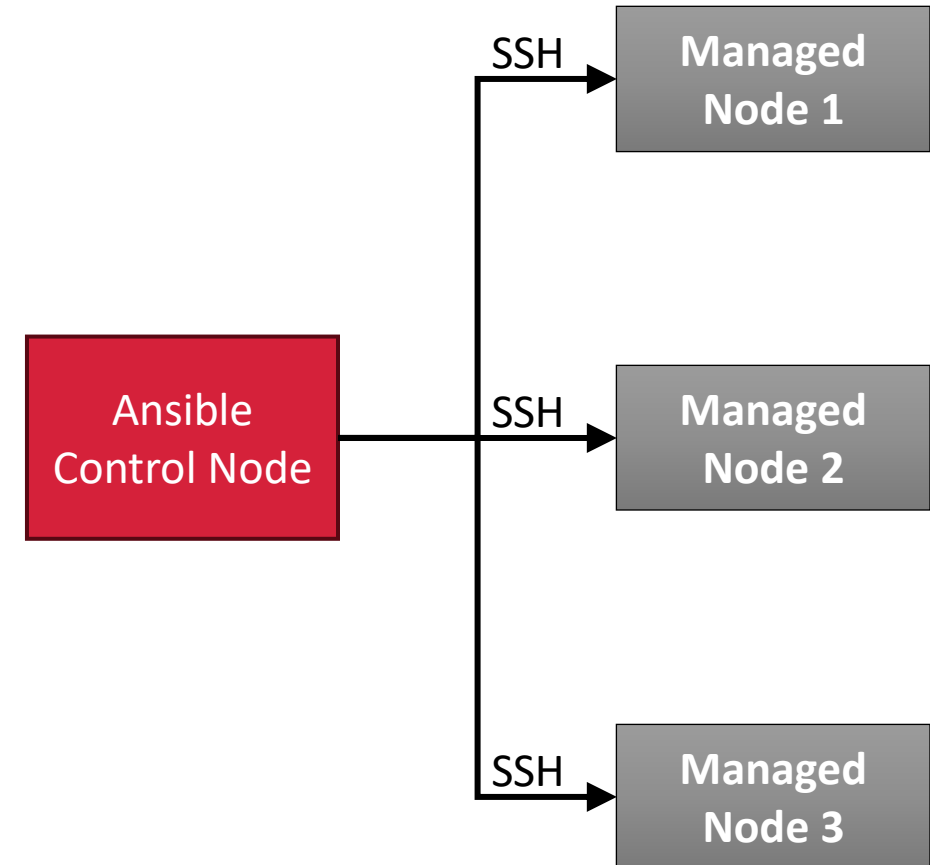
This Photo by Unknown Author is licensed under CC BY-SA

SSH – recap (con't)

Until now, we have used our laptops as clients to connect to EC2 servers.

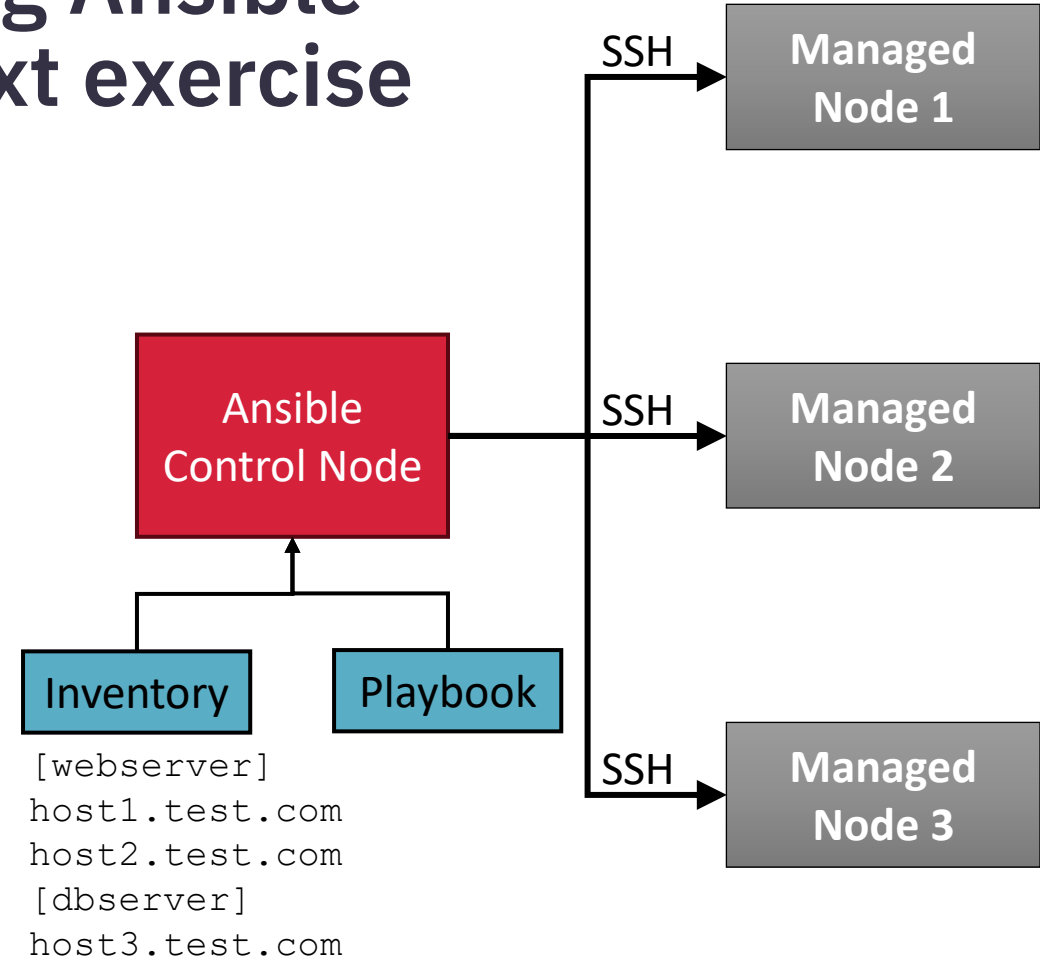
Here, we need to use an EC2 as a client [the control node] to connect to multiple other EC2s [the managed nodes].

To achieve this outcome, we will create the SSH keys as part of the configuration of the control node and add this public key to each of the managed nodes. This will be automated to a great extent.



Managing Multiple Servers using Ansible – A high-level overview of the next exercise

1. Use control node repo * to create the node that will manage the servers created by next step. This will include creation of SSH keys, with the public key included in the ansible output.
2. Create a key file using this output and update the variables file with a reference to this key on your laptop. This key will be copied to all EC2s created by the managed nodes repo **.
3. Use the control node to apply updates on the managed nodes using Ansible.



* https://github.com/SMannionYorkU/CloudOps_Module4_Class6_Repo2_ProvisionControlNode.git

** https://github.com/SMannionYorkU/CloudOps_Module4_Class6_Repo3_ProvisionManagedNodes.git

Managing Multiple Servers using Ansible (1 of 16)

Please execute the following steps to complete this exercise

1. Create the control node using ProvisionControlNode terraform code. While the EC2 is provisioned, review the content of the `install_control_node.yaml` file

- This Ansible file creates SSH keys on this server and outputs the public key during the playbook step of the terraform execution.
- Note `become` is false at the top of the file, it is important that the key is created as the `ec2-user` as it is this user that we will connect to the control nodes, not the root user.

```
Class6 > 01.CreateControlNode > ! install_control_node.yaml
1  ---
2  - name: Create control node
3    hosts: localhost
4    remote_user: ec2-user
5    become: false # want to remain as ec2-user, not root
6
7    tasks:
8      - name: generate SSH key
9        openssh_keypair:
10          path: ~/.ssh/id_rsa
11          type: rsa
12          size: 4096
13          state: present
14          force: no
15
16      - name: shell to create table and import data
17        shell: cat ~/.ssh/id_rsa
18        register: cat_output
19
20      - name: Display the output
21        debug:
22          var: cat_output.stdout
23
```

Managing Multiple Servers using Ansible (3 of 16)

3. Check the output from the terraform execution, specifically the section that relates to the public key output.

- Highlight and copy the public key, we will use it in creating the managed nodes, which will allow the control node to connect to these managed nodes using Ansible over SSH.

4. Clone the ProvisionManagedNodes git repo. Add a file to the root of this repo called id_rsa.pub and copy the text from the previous step into this file. .gitignore is used to ensure that this file is not checked into git, as this key should not be shared.

```
aws_instance.example (remote-exec): PLAY [Create control node] *****
*****

aws_instance.example (remote-exec): TASK [Gathering Facts] *****
*****
aws_instance.example (remote-exec): ok: [localhost]

aws_instance.example (remote-exec): TASK [generate SSH key] *****
*****
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [output the public key so that it can be used in creating the managed nodes] ***
aws_instance.example (remote-exec): changed: [localhost]

aws_instance.example (remote-exec): TASK [Display the output] *****
*****
aws_instance.example (remote-exec): ok: [localhost] => {
aws_instance.example (remote-exec):   "cat_output.stdout": "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDd4nYiI4
yyTBjI9IoI+07X+HqZZ1LqxILNjftnzwr1619rElNvjIxMcIOfBfARbk+LakIMGy+oTasSgDcBiOoKQuYcFRq4saWwyZNidzbCjeHfuHNKGL
eDL31uVw7YuWgmCBSt4W9ewuerZy3NBfZgQZC+euLqqVi8UuKxXR63tfvBYM9HVY5AJrfZjwbkxeGYMw2vI7w1CCKyjcbltrpmT/we1mop
uqpu3j2djUHLpoBweeOm92I0pPxBo7uAsmnebhrSR8V+60cxb5dVUocWM19UNZMYfq5V7RJ6WMTWd7cYAWM5qTw0QtzmByZot7nT9dLFv
bna24UPuHI7fs03t05RrnIBJ6Smc/QB0q3zfhw0Mc2REW2iTGbJ5L6+L1F3eVlZqkAzMUX8GmIQiwpKvtXbYx80a9k77EP1SYex5ZGwP9CDI
HcKvhQtmSCuzVmdB6Yo+6gchRbjHd3nPKScQM1Jns3Zm2TnctyKrIE14DUQsEK84WVF5Pc6p2prdodbw1sC/w5p513N30uQEIXuDcqA0L5Yz
92beEc4WrB1cJnPkWe+trdySR9dP3IqUMNLgxFFM6XtWeN3e15EFdbEKuwQpru11IthU01Rkwubb3z66j3bm9VdJf0/OrkxbmNX8QT0V0ord
fxxIayU1FwdOpVEz9GfQQA0IA8Q== "
aws_instance.example (remote-exec): }

aws_instance.example (remote-exec): PLAY RECAP *****
*****
aws_instance.example (remote-exec): localhost                                : ok=4    changed=2    unreachable=0    failed=0
d=0    skipped=0    rescued=0    ignored=0

aws_instance.example: Creation complete after 2m36s [id=i-0575e8d222d52333b]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:
Ansible-ControlNode-PublicIP = "54.236.87.227"
```

Managing Multiple Servers using Ansible (4 of 16)

5. Reviewing the CreateManagedNodes codebase, you will notice it has the familiar module structure from previous terraform exercises.
6. One new thing to note is the use of the count keyword in the modules/compute/main.tf file. This is in the resource "aws_instance" "managed_node" code and indicates that we need 3 instance of this server.
7. IP addresses for these servers are output as an array, passed from the compute module to the root output.

```
main.tf x
Class6 > 01.CreateManagedNodes > modules > compute > main.tf > resource "aws_instance"
21 #Create and bootstrap Managed Node Servers
22 #=====
23 resource "aws_instance" "managed_node" {
24   # Creates 3 identical aws ec2 instances
25   count = 3
26
27   instance_type = "t2.micro"
28   ami = data.aws_ssm_parameter.webserver-ami.value
29   tags = {
30     Name = "host_node_${count.index}"
31   }
32   key_name = aws_key_pair.aws-key.key_name
33   associate_public_ip_address = true
34   vpc_security_group_ids = [var.security_group]
35   subnet_id = var.subnets
36
37 }
```

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

```
Managed-Nodes = [
  "3.89.116.157",
  "3.87.116.143",
  "3.80.63.157",
]
```

Managing Multiple Servers using Ansible (5 of 16)

8. Let's check connectivity before we go any further.

- ✓ From your laptop, SSH to the control node (IP found in output from CreateControlNode).

This should work.

- ✗ Next from your laptop, SSH to one of the managed nodes (IPs found in array in output from CreateManagedNodes). This should fail.

- ✓ Finally, from the control node, SSH to one of the managed nodes (IPs found in array in output from CreateManagedNodes). This should work.

```
[shane@Shanes-MacBook-Pro ~ % ssh ec2-user@54.236.87.227
The authenticity of host '54.236.87.227 (54.236.87.227)' can't be established.
ED25519 key fingerprint is SHA256:cz471hdBy0/ow3biauvRJ+FwDovyL5+egrPsKR+zZKc.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.236.87.227' (ED25519) to the list of known hosts.
Last login: Wed Jul 26 01:33:28 2023 from 99.240.116.149
```

```
--|  --|_ )
_| (  /   Amazon Linux 2 AMI
---|\---|---
```

```
https://aws.amazon.com/amazon-linux-2/
No packages needed for security; 2 packages available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-172-31-89-142 ~]$
```

```
[shane@Shanes-MacBook-Pro ~ % ssh ec2-user@3.89.116.157
The authenticity of host '3.89.116.157 (3.89.116.157)' can't be established.
ED25519 key fingerprint is SHA256:PEyPjPxjz0UtEBDopX8gMICqHLK/Y0c2oTOWcWmeKRw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.89.116.157' (ED25519) to the list of known hosts.
ec2-user@3.89.116.157: Permission denied (publickey,gssapi-keyex,gssapi-with-mic
).
shane@Shanes-MacBook-Pro ~ %
```

```
[ec2-user@ip-172-31-89-142 ~]$ ssh ec2-user@3.89.116.157
The authenticity of host '3.89.116.157 (3.89.116.157)' can't be established.
ECDSA key fingerprint is SHA256:AkaWT03jCc3GxxHp88z2Z4/nUV3oiXLdIN/EEEr8ZY4.
ECDSA key fingerprint is MD5:0b:bf:d1:c0:79:70:c9:28:85:6e:56:62:e5:50:e5:29.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '3.89.116.157' (ECDSA) to the list of known hosts.
```

```
--|  --|_ )
_| (  /   Amazon Linux 2 AMI
---|\---|---
```

```
https://aws.amazon.com/amazon-linux-2/
3 package(s) needed for security, out of 12 available
Run "sudo yum update" to apply all updates.
```

Managing Multiple Servers using Ansible (6 of 16)

9. With this approach, only the control node communicates with the managed nodes, not the developer's laptop.
10. Next step is to create an inventory file. With 3 managed nodes created, we configure 2 as webserver and 1 as a database server.
 - Use the IP addresses for the managed nodes in the output from the 2nd terraform and create an inventory file hosts.ini in /home/ec2-user

Outputs:

```
Managed-Nodes = [  
  "3.89.116.157",  
  "3.87.116.143",  
  "3.80.63.157",  
]
```

```
[webserver]  
3.89.116.157  
3.87.116.143  
[  
[database]  
3.80.63.157
```

Managing Multiple Servers using Ansible (7 of 16)

11. Start with an ad hoc command to connect to all managed nodes in the hosts.ini file.

```
ansible all -m shell -a "free -m" -i hosts.ini
```

12. As this is the first connection, you will need to enter yes to accept the fingerprint for each node. You will also see a python warning which we will fix.

13. Edit the ansible.cfg file using this command

```
sudo vi /etc/ansible/ansible.cfg
```

14. Add the following line in the [defaults] section

```
interpreter_python = auto_silent
```

```
[[ec2-user@ip-172-31-89-142 ~]$ ansible databases -m shell -a "free -m" -i hosts.ini
[WARNING]: Platform linux on host 3.80.63.157 is using the discovered Python interpreter
at /usr/bin/python, but future installation of another Python interpreter could change
this. See
https://docs.ansible.com/ansible/2.9/reference_appendices/interpreter_discovery.html for
more information.
3.80.63.157 | CHANGED | rc=0 >>
      total      used      free      shared  buff/cache   available
Mem:        970      92       311         0         566         731
Swap:         0         0         0
```

```
shane — ec2-user@ip-172-31-89-142:~ — ssh ec2-user@54.236.87.227 — 91x29

# config file for ansible -- https://ansible.com/
# =====

# nearly all parameters can be overridden in ansible-playbook
# or with command line flags. ansible will read ANSIBLE_CONFIG,
# ansible.cfg in the current working directory, .ansible.cfg in
# the home directory or /etc/ansible/ansible.cfg, whichever it
# finds first

[defaults]

# some basic default values...
interpreter_python = auto_silent
#inventory      = /etc/ansible/hosts
#library        = /usr/share/my_modules/
#module_utils   = /usr/share/my_module_utils/
#remote_tmp     = ~/.ansible/tmp
#local_tmp      = ~/.ansible/tmp

[[ec2-user@ip-172-31-89-142 ~]$ ansible databases -m shell -a "free -m" -i hosts.ini
3.80.63.157 | CHANGED | rc=0 >>
      total      used      free      shared  buff/cache   available
Mem:        970      92       311         0         566         731
Swap:         0         0         0
```


Managing Multiple Servers using Ansible (8 of 16)

15. Verify that the python warning is resolved.

16. You can use the hosts file to run commands across all nodes in the hosts.ini, or target specific files based on the groups to which they belong, which is a very useful feature.

```
ansible databases -m shell -a "free -m" -i hosts.ini
```

```
ansible webservers -m shell -a "free -m" -i hosts.ini
```

```
[[ec2-user@ip-172-31-89-142 ~]$ ansible databases -m shell -a "free -m" -i hosts.ini
3.80.63.157 | CHANGED | rc=0 >>
      total        used          free      shared  buff/cache   available
Mem:      970         92          309           0         568         731
Swap:      0           0            0
[[ec2-user@ip-172-31-89-142 ~]$ ansible webservers -m shell -a "free -m" -i hosts.ini
3.89.116.157 | CHANGED | rc=0 >>
      total        used          free      shared  buff/cache   available
Mem:      970         91          310           0         568         732
Swap:      0           0            0
3.87.116.143 | CHANGED | rc=0 >>
      total        used          free      shared  buff/cache   available
Mem:      970         91          306           0         572         732
Swap:      0           0            0
```

Managing Multiple Servers using Ansible (9 of 16)

11. The inventory file can contain overlapping groups too, this does not create conflicts. Edit the hosts.ini file to add two new groups.

12. Run an ad hoc command for the South American servers.

There is no conflict, even though one is a webserver and the other is a database.

`ansible southamerica -m shell -a "free -m" -i hosts.ini`

```
[webservers]
3.89.116.157
3.87.116.143

[databases]
3.80.63.157

[northamerica]
3.89.116.157

[southamerica]
3.87.116.143
3.80.63.157
```

```
[[ec2-user@ip-172-31-89-142 ~]$ ansible southamerica -m shell -a "free -m" -i hosts.ini
3.80.63.157 | CHANGED | rc=0 >>
              total    used         free      shared  buff/cache   available
Mem:           970        92          309           0         569         731
Swap:            0           0            0
3.87.116.143 | CHANGED | rc=0 >>
              total    used         free      shared  buff/cache   available
Mem:           970        91          306           0         572         732
Swap:            0           0            0
```

Managing Multiple Servers using Ansible (10 of 16)

13. This approach can also be used with playbooks. To start with a clean slate, you can destroy and recreate the control nodes. The SSH key does not change if the control node remains, so the only update will be to the hosts.ini on the control node.
14. Clone the code in the class ControlNodeFiles repo* and scp into the home directory for the ec2-user on the control node. You will recognise the install_httpd.yaml and install_mariadb.yaml files from earlier. Let's compare the versions here with the originals.

Outputs:

```
Managed-Nodes = [  
  "107.23.180.44",  
  "3.93.60.77",  
  "34.229.15.76",  
]
```

Terraform output

```
[webservers]  
107.23.180.44  
34.229.15.76  
  
[databases]  
3.93.60.77  
  
[northamerica]  
3.93.60.77  
34.229.15.76  
  
[southamerica]  
107.23.180.44
```

Hosts.ini

Managing Multiple Servers using Ansible (11 of 16)

15. Differences are as follows:

Hosts previously was localhost, now updated to webservers to reference this group in hosts.ini.

Zip file must be copied to managed webserver for install, previously this was a local operation

For the copy, we need to indicate the files are already on the managed node, not copied from the control node

```
1 ---
2 - name: Install httpd
3 - hosts: localhost
4   remote_user: ec2-user
5   become: true
6
7   tasks:
8     - name: install yum httpd package
9       yum:
10        name: httpd
11        state: present
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
```

```
1 ---
2 - name: Install httpd
3+ hosts: webservers
4   remote_user: ec2-user
5   become: true
6
7   tasks:
8     - name: install yum httpd package
9       yum:
10        name: httpd
11        state: present
12
13+ - name: copy zip file to remote server
14+   copy:
15+     src: /home/ec2-user/Einstein.zip
16+     dest: /home/ec2-user/Einstein.zip
17+
18 - name: Unarchive a file that is already on the managed node
19   unarchive:
20     src: /home/ec2-user/Einstein.zip
21     dest: /tmp
22     remote_src: yes
23
24 - name: Copy files
25   copy:
26     src: '/tmp/{{item}}'
27     dest: /var/www/html/
28     owner: ec2-user
29+   remote_src: true
30
31   loop:
32     - index.html
33     - einstein.jpg
34
35 - name: Start and Enable httpd
36   systemd:
37     name: httpd
38     state: started
39     enabled: true
```

Managing Multiple Servers using Ansible (12 of 16)

16. Before running the playbook, it is a good idea to connect to the managed node over SSH from the control node, so that the fingerprint can be recorded outside the ansible process.

17. Once done, the following command will execute the playbook, targeting the webserver group from the included hosts.ini.

```
ansible-playbook -i hosts.ini install_httpd.yaml
```

18. Inspect the output, noting that the IPs match the webserver group from hosts.ini, which is specified by **hosts:** in install_httpd.yaml.

```
[ec2-user@ip-172-31-89-142 ~]$ ansible-playbook -i hosts.ini install_httpd.yaml
```

```
PLAY [Install httpd] *****

TASK [Gathering Facts] *****
ok: [107.23.180.44]
ok: [34.229.15.76]

TASK [install yum httpd package] *****
changed: [107.23.180.44]
changed: [34.229.15.76]

TASK [copy zip file to remote server] *****
changed: [34.229.15.76]
changed: [107.23.180.44]

TASK [Unarchive a file that is already on the remote machine] *****
changed: [34.229.15.76]
changed: [107.23.180.44]

TASK [Copy files] *****
changed: [34.229.15.76] => (item=index.html)
changed: [107.23.180.44] => (item=index.html)
changed: [107.23.180.44] => (item=einstein.jpg)
changed: [34.229.15.76] => (item=einstein.jpg)

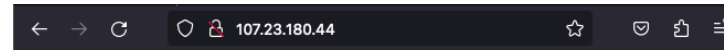
TASK [Start and Enable httpd] *****
changed: [107.23.180.44]
changed: [34.229.15.76]

PLAY RECAP *****
107.23.180.44      : ok=6    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
34.229.15.76     : ok=6    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Managing Multiple Servers using Ansible (13 of 16)

19. Check both webserver IPs using your browser.

20. Next, we will run the database playbook, but first review the differences between the previous version and this one.



Date, Time, and Server Name

Current Date and Time: 2023-07-27, 11:12:33 p.m.

Server Name: 107.23.180.44

Top 10 Albert Einstein Quotes

"Imagination is more important than knowledge."

"The only source of knowledge is experience."

"Strive not to be a success, but rather to be of value."

"The true sign of intelligence is not knowledge but imagination."

"Logic will get you from A to B. Imagination will take you everywhere."

"The difference between stupidity and genius is that genius has its limits."

"Education is what remains after one has forgotten what one has learned in school."

"In the middle of difficulty lies opportunity."

"Look deep into nature, and then you will understand everything better."

"Imagination is everything. It is the preview of life's coming attractions."



Date, Time, and Server Name

Current Date and Time: 2023-07-27, 11:12:58 p.m.

Server Name: 34.229.15.76

Top 10 Albert Einstein Quotes

"Imagination is more important than knowledge."

"The only source of knowledge is experience."

"Strive not to be a success, but rather to be of value."

"The true sign of intelligence is not knowledge but imagination."

"Logic will get you from A to B. Imagination will take you everywhere."

"The difference between stupidity and genius is that genius has its limits."

"Education is what remains after one has forgotten what one has learned in school."

"In the middle of difficulty lies opportunity."

"Look deep into nature, and then you will understand everything better."

"Imagination is everything. It is the preview of life's coming attractions."



Managing Multiple Servers using Ansible (14 of 16)

21. Differences are as follows:

Hosts previously was localhost, now updated to databases to reference this group in hosts.ini.

EPEL repo is added and registered through ansible, previously through terraform.

Sql file must be copied to managed database server for execution, previously this was a local operation

```
1 ---
2 - name: Install mariadb
3 - hosts: localhost
4 - remote_user: ec2-user
5 - become: true
6
7 - tasks:
8   - name: install mariadb
9     yum:
10      - name: mariadb-server
11        state: present
12
13   - name: Start and Enable Mariadb
14     systemd:
15      - name: mariadb
16        state: started
17        enabled: true
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

```
1 ---
2 - name: Install mariadb
3 - hosts: databases
4 - remote_user: ec2-user
5 - become: true
6
7 - tasks:
8   - name: install mariadb
9     yum:
10      - name: mariadb-server
11        state: present
12
13   - name: Start and Enable Mariadb
14     systemd:
15      - name: mariadb
16        state: started
17        enabled: true
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```


Managing Multiple Servers using Ansible (15 of 16)

22. The following command will execute the playbook, targeting the database group from the included hosts.ini.

```
ansible-playbook -i hosts.ini install_mariadb.yaml
```

23. Inspect the output, noting that the IP matches the database group from hosts.ini, which is specified by **hosts:** in install_httpd.yaml.

```
[ec2-user@ip-172-31-89-142 ~]$ ansible-playbook -i hosts.ini install_mariadb.yaml

PLAY [Install mariadb] *****

TASK [Gathering Facts] *****
ok: [3.93.60.77]

TASK [install mariadb] *****
changed: [3.93.60.77]

TASK [Start and Enable Mariadb] *****
changed: [3.93.60.77]

TASK [shell to create table and import data] *****
changed: [3.93.60.77]

TASK [install MySQL client library to allow use of mysql_user] *****
changed: [3.93.60.77]

TASK [Create a new database with name 'bobdata'] *****
changed: [3.93.60.77]

TASK [create user] *****
[WARNING]: Module did not set no_log for update_password
changed: [3.93.60.77]

TASK [copy zip file to remote server] *****
changed: [3.93.60.77]

TASK [shell to create table and import data] *****
changed: [3.93.60.77]

PLAY RECAP *****
3.93.60.77 : ok=9    changed=8    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Managing Multiple Servers using Ansible (16 of 16)

24. Connect to database server
and verify that the playbook
has configured the server as
expected.

```
[ec2-user@ip-10-0-1-111 ~]$ mysql -u Bob -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 5.5.68-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases
+-----+
| Database |
+-----+
| information_schema |
| bobdata |
| mysql |
| performance_schema |
| test |
+-----+
5 rows in set (0.00 sec)

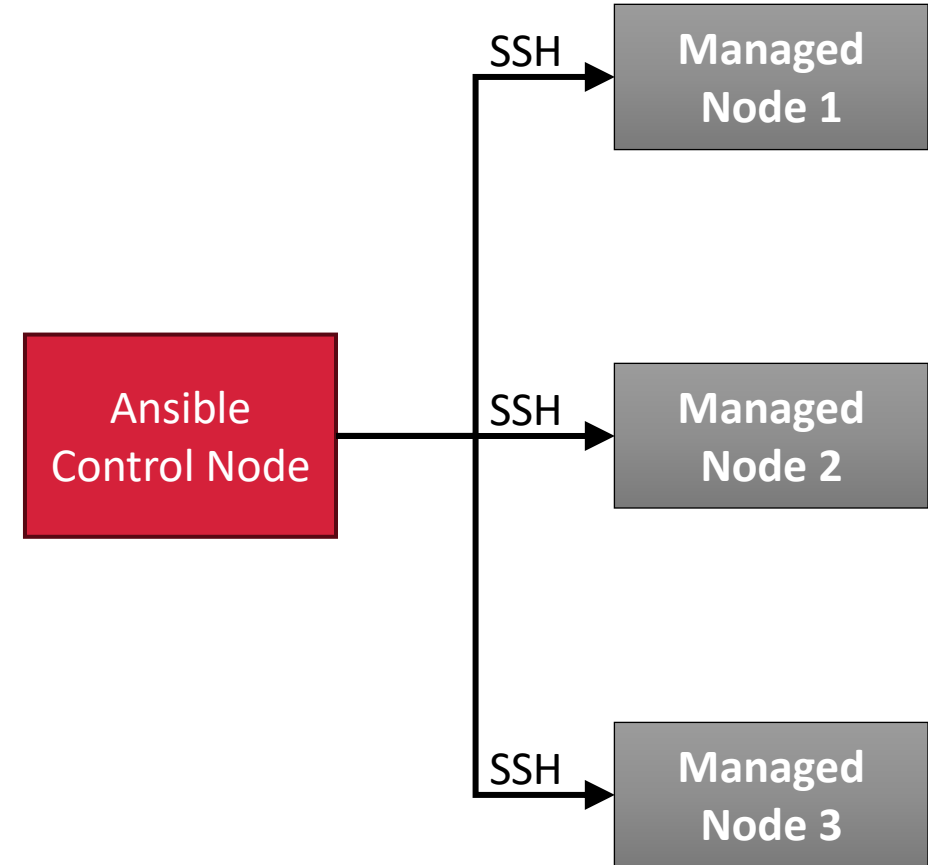
MariaDB [(none)]> use bobdata;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [bobdata]> show tables;
+-----+
| Tables_in_bobdata |
+-----+
| SteveJobsQuotes |
+-----+
1 row in set (0.00 sec)
```

```
MariaDB [bobdata]> select * from SteveJobsQuotes;
+-----+-----+
| id | quote |
+-----+-----+
| 1 | Stay hungry, stay foolish. |
| 2 | Innovation distinguishes between a leader and a follower. |
| 3 | Your work is going to fill a large part of your life, and the only way to be truly satisfied is to do what you believe is great work. |
| 4 | The people who are crazy enough to think they can change the world are the ones who do. |
| 5 | Remembering that you are going to die is the best way I know to avoid the trap of thinking you have something to lose. You are already naked. There is no reason not to follow your heart. |
| 6 | Innovation comes from people meeting up in the hallways or calling each other at 10:30 at night with a new idea or because they realized something that shoots holes in how we've been thinking about a problem. |
```

Managing Multiple servers using Ansible - Conclusion

- From the previous slides you can see the power of Ansible, how versatile and reliable it can be and how it creates consistency across the servers you manage, while making it much easier to maintain that it has been previously.
- This concludes this section on managing multiple servers using Ansible.
- Let's now look at managing deployments using Ansible.



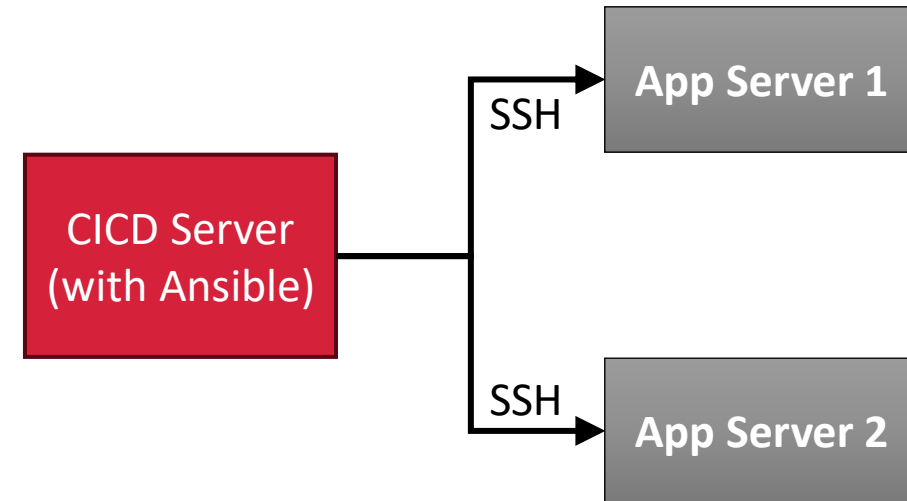
Ansible Deployments

Ansible Deployments (con't)

A common use case for Ansible is to deploy code as part of a CD pipeline.

Benefits of Ansible for this purpose:

- SSH connection from control server to multiple servers
- Range of commands available within playbook such as stop/start services, run updates, copy files that support deployment.
- Consistency that is achieved through Ansible across multiple related servers e.g., load balanced.
- We will look at this in more detail as part of Jenkins in coming weeks.



Next, Ansible Roles

Ansible Roles

- As your playbooks become more complex, Ansible roles can help with reuse and sharing code, similar to modules in Terraform.
- Roles provides a clear framework to structure tasks, variables, handlers, metadata, templates, and other files.
- You can create a role structure using the command
ansible-galaxy init <yourRoleName>
- ansible-galaxy is bundled as part of your Ansible install.

```
[[ec2-user@ip-172-31-89-142 Ansible]$ ansible-galaxy init myCloudOpsRole
- Role myCloudOpsRole was created successfully
[[ec2-user@ip-172-31-89-142 Ansible]$ ls -lrt
total 0
drwxrwxr-x 10 ec2-user ec2-user 154 Jul 29 11:22 myCloudOpsRole
[[ec2-user@ip-172-31-89-142 Ansible]$ cd myCloudOpsRole/
[[ec2-user@ip-172-31-89-142 myCloudOpsRole]$ ls -lrt
total 4
drwxrwxr-x 2 ec2-user ec2-user  6 Jul 29 11:22 templates
-rw-rw-r-- 1 ec2-user ec2-user 1328 Jul 29 11:22 README.md
drwxrwxr-x 2 ec2-user ec2-user  6 Jul 29 11:22 files
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 handlers
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 defaults
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 tasks
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 meta
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 vars
drwxrwxr-x 2 ec2-user ec2-user 39 Jul 29 11:22 tests
```


Ansible Roles (con't)

- To use roles, you must include at least one of the folders that are created by the `init` command in the screenshot. Our focus with playbooks has been tasks, so that would be a good place to start.
- Through the rest of the structure, you can define handlers for different activities, define default values for variables and add any files (such as Einstein.html from our previous exercises).
- Further reading on this topic is recommended so that you have a good understanding of what is involved. *

```
[ec2-user@ip-172-31-89-142 Ansible]$ ansible-galaxy init myCloudOpsRole
- Role myCloudOpsRole was created successfully
[ec2-user@ip-172-31-89-142 Ansible]$ ls -lrt
total 0
drwxrwxr-x 10 ec2-user ec2-user 154 Jul 29 11:22 myCloudOpsRole
[ec2-user@ip-172-31-89-142 Ansible]$ cd myCloudOpsRole/
[ec2-user@ip-172-31-89-142 myCloudOpsRole]$ ls -lrt
total 4
drwxrwxr-x 2 ec2-user ec2-user  6 Jul 29 11:22 templates
-rw-rw-r-- 1 ec2-user ec2-user 1328 Jul 29 11:22 README.md
drwxrwxr-x 2 ec2-user ec2-user  6 Jul 29 11:22 files
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 handlers
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 defaults
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 tasks
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 meta
drwxrwxr-x 2 ec2-user ec2-user 22 Jul 29 11:22 vars
drwxrwxr-x 2 ec2-user ec2-user 39 Jul 29 11:22 tests
```

* <https://spacelift.io/blog/ansible-roles>
https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html

Lab 2

1. Clone all the code used in the last exercise [Managing Multiple Servers using Ansible].
2. Modify your copy of the code to achieve the following:
 - create two additional webserver in managed nodes
 - configure the Einstein webpage on two webserver
 - configure the Cloud Strategy webpage on the other two webserver
 - configure the database server as per existing

As all 4 webserver will have common configuration for httpd install, this could be achieved with a dedicated httpd playbook that runs on all webserver

Separate playbooks for Einstein and Cloud Strategy deployments, that are executed on their respective webserver, is recommended.

Submit a report with the following information

- GitHub url for your completed work
- Screenshot of hosts.ini
- Screenshots of output from execution of each Ansible playbook on the control node.
- Screenshots of browser (including clear view of IP in address bar) showing Einstein and Cloud Strategy pages loaded

Journey Map

